

```
@SpringBootApplication
public class UserAddService {
    public static void main(String[] args) {
        SpringApplication.run(UserAddService.class, args);
    }
}
```

Configuration

Apart from the code we have seen so far, we also need to supply configuration. Though Spring Boot prefers convention over configuration, it works only for defaults. Whenever we want to override the defaults, we need to supply those overrides as configurations in the *application.properties* file. Here is the configuration for *AddService*:

```
spring.datasource.url=jdbc:h2:mem:glarimy
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

This configuration enables a Web based GUI console to interact with the H2 database. It also indicates the hibernate DDL processor to create a database named *glarimy* as per the annotations.

That's all.

A well-designed Spring Boot application thus requires these basic constructs: domain objects, repositories, DTOs, and controllers. This is exactly what domain-driven design proposed, isn't it? With the powerful annotations of Spring Boot, we get all the technological support like REST, persistence, validations, etc, without deviating from this design.

Building and running the microservice

Let us build the microservice and package it with the following command:

```
mvn clean package
```

It generates a fat JAR named *ums-add-service-0.0.1-SNAPSHOT.jar* under the *target* folder. Run the application using the usual Java command. You don't require any special skills and tools to run the microservice.

```
java -jar target/ums-add-service-0.0.1-SNAPSHOT.jar
```

Observe the console, and wait till the service is up and running. Since we do not have any Web interface for this service, you can use CURL to post a new user to the system.

However, just before doing that, verify the database by opening the link to the H2 database console at the URL *http://localhost:8080/h2-console*. Enter *jdbc:h2:mem:glarimy* as the JDBC URL in the dialogue and click *Connect*. Find the database table named *UMS_USERS* and click on it. It displays

the query *SELECT * from UMS_USERS*. You do not get any records by running the query as we are yet to post any users.

Now open a terminal and run the *CURL* command to add a new user:

```
curl -X POST -H 'Content-Type: application/json' -i http://localhost:8080/user --data '{"name":"Krishna Mohan", "phone":"9731423166"}'
```

You should receive a *success* response. You can also check the database console to find the newly added record, as shown in Figure 5.

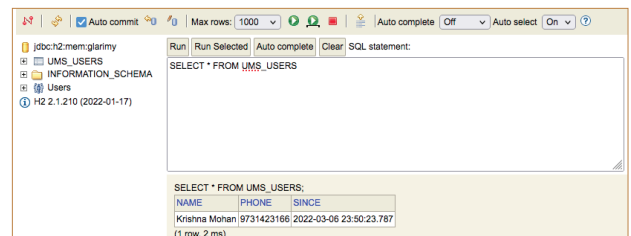


Figure 5: Results in the H2 database console

Road ahead

This part of 'The Design Odyssey' series illustrates a way to build microservices on the Java platform using the Spring Boot framework. It demonstrates that microservices are essentially modelled based on the concepts we have covered under domain-driven design. And the fundamental object-oriented patterns are sprinkled all over, thanks to the plumbing done by the framework. The biggest difference between a traditional Java Web application and a modern Java microservice is that we no longer package them as WAR files; instead, we package them as standard JAR files with an embedded Web server.

However, the Spring Boot microservice cannot be termed as self-sufficient yet, as it still requires a JRE pre-installed. Installing a JRE on every machine where the microservice is to be deployed is quite a task. How about including the JRE also into the deployment unit so that it truly becomes self-sufficient?

That's exactly what containerisation is. In the next part of this series of articles, we will containerise the Spring Boot service using Dockers and deploy it on a Kubernetes cluster. **END** 🐧

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.